PriceScope

Distributed Intelligent Product Comparison Platform

PROJECT REPORT

An Enterprise-Grade Distributed Computing Platform using
Microservices, FastAPI, Redis, Celery, MCP, Selenium, AsyncIO and Distributed Task Scheduling

Submitted By :
Ayan Kumar Batabyal
Ashish Kumar samantary
Sayan Goswami

Department :
Computer Science

Project Domain :
Distributed Computing and Intelligent Systems

Technologies Used :
Python, FastAPI, Redis, Celery, Selenium, MCP, AsyncIO

Academic Year :
2026

Ramakrishna Mission Vivekananda Educational and Research Institute

Abstract

The rapid growth of e-commerce platforms has created a highly distributed digital marketplace where identical products are available across multiple vendors with varying prices, ratings, seller information, and delivery conditions. Traditional product comparison systems are often centralized, inefficient, and incapable of handling concurrent distributed data aggregation in real time.

This project presents **PriceScope**, an enterprise-grade distributed product comparison platform designed using modern distributed computing principles and scalable microservices architecture. The system integrates asynchronous APIs, intelligent orchestration, distributed task execution, parallel processing, real-time scraping, and fault-tolerant service communication to provide efficient multi-platform product aggregation.

PriceScope consists of autonomous marketplace microservices for Amazon, eBay, Google Shopping, Walmart, and Target. Each service independently performs distributed product retrieval while communicating asynchronously through a centralized API Gateway. The platform utilizes FastAPI for high-performance asynchronous APIs, Selenium for dynamic browser automation, BeautifulSoup for parsing, Redis as a distributed message broker, Celery for asynchronous distributed task scheduling, and MCP-based orchestration for intelligent normalization and ranking.

The architecture demonstrates multiple distributed computing concepts including:

- Distributed Microservices Architecture
- Parallel Asynchronous Execution
- Distributed Task Scheduling
- Event-Driven Communication
- Service Isolation
- Fault Tolerance
- Horizontal Scalability
- Intelligent Orchestration

Experimental analysis demonstrates successful concurrent service execution, scalable asynchronous communication, distributed workload management, and efficient response aggregation. The project closely resembles real-world cloud-native enterprise distributed systems.

Keywords : Distributed Computing, Microservices, FastAPI, Redis, Celery, AsyncIO, MCP, Selenium, Distributed Systems, Event-Driven Architecture, Enterprise Systems.

Contents

1 Introduction **3**

2 Problem Statement **3**

3 Objectives **3**

4 Enterprise Distributed Marketplace Services **4**

5 System Architecture **4**
5.1 Architecture Characteristics 4
5.2 System Architecture Diagram 5

6 Distributed Execution Workflow **5**

7 Distributed Computing Concepts Used **6**
7.1 Microservices Architecture 6
7.2 Asynchronous Processing 6
7.3 Parallel Execution 6
7.4 Distributed Task Queue 6
7.5 Fault Tolerance 6
7.6 Horizontal Scalability 6
7.7 Service Orchestration 6

8 Technology Stack **7**

9 Implementation Details **7**
9.1 API Gateway 7
9.2 Amazon Service 7
9.3 Google Shopping Service 7
9.4 eBay Service 7
9.5 Walmart Service 7
9.6 Target Service 7
9.7 Redis and Celery 8

10 Results and Performance Analysis **8**
10.1 Experimental Evaluation 8
10.2 Observed Technical Outcomes 8

11 Advantages **9**

12 Limitations **9**

13 Future Enhancements **9**

14 Conclusion **9**

15 References **10**

1 Introduction

Modern e-commerce ecosystems contain enormous volumes of distributed product information across multiple platforms such as Amazon, eBay, Walmart, Google Shopping, and Target. Consumers are required to manually compare prices, ratings, seller reliability, and delivery conditions across several platforms before making purchasing decisions.

Traditional monolithic comparison systems suffer from:

- High response latency
- Scalability limitations
- Poor fault tolerance
- Centralized bottlenecks
- Inefficient data aggregation

Distributed computing architectures solve these challenges by dividing workloads across multiple independent services capable of executing concurrently.

PriceScope is designed as a distributed intelligent product comparison platform implementing:

- Distributed Microservices
- Parallel Execution
- Asynchronous APIs
- Distributed Task Queues
- Event-Driven Communication
- Intelligent Product Normalization
- Enterprise Service Orchestration

The platform aggregates product data concurrently from multiple marketplace services and returns optimized comparison results through a centralized orchestration layer.

2 Problem Statement

Online consumers face several difficulties while comparing products across multiple e-commerce platforms:

- Manual comparison across websites
- Dynamic and inconsistent pricing
- Different product naming conventions
- Lack of centralized real-time aggregation
- Slow response time in traditional systems
- Poor scalability in monolithic architectures

Existing systems are unable to efficiently support:

1. Concurrent multi-platform product retrieval
2. Distributed asynchronous execution
3. Scalable workload processing
4. Intelligent product normalization
5. Enterprise-grade orchestration

Therefore, a scalable distributed intelligent comparison system is required.

3 Objectives

- Design a distributed microservices-based comparison platform.
- Implement independent marketplace services.
- Enable asynchronous parallel execution.
- Integrate Redis and Celery for distributed task scheduling.

- Develop intelligent orchestration mechanisms.
- Demonstrate real-world distributed computing concepts.
- Build a scalable and fault-tolerant architecture.

4 Enterprise Distributed Marketplace Services

PriceScope integrates multiple autonomous marketplace microservices operating inside a distributed asynchronous ecosystem.

Microservice	Port	Technical Functionality
Amazon Service	8005	Selenium-powered distributed browser automation service for dynamic product extraction.
eBay Service	8003	Asynchronous distributed product retrieval and parsing service.
Google Shopping Service	8004	SerpAPI-based scalable distributed search aggregation service.
Walmart Service	8006	Hybrid distributed scraping and asynchronous product extraction service.
Target Service	8007	Enterprise-grade distributed Target marketplace integration service.
MCP Server	8010	Intelligent orchestration server for semantic ranking and normalization.
API Gateway	8000	Centralized orchestration and distributed communication layer.
Redis Server	6379	Distributed cache and message broker supporting event-driven communication.
Celery Workers	Dynamic	Enterprise asynchronous distributed task execution system.
Frontend Interface	Browser	Visualization and user interaction layer.

Table 1: Enterprise Distributed Marketplace Services

5 System Architecture

PriceScope follows a distributed service-oriented architecture designed for scalability, resilience, and parallel computation.

5.1 Architecture Characteristics

- Decentralized Service Execution
- Loose Coupling Between Services
- Independent Deployment Capability
- Event-Driven Distributed Processing
- Parallel Asynchronous Communication
- Fault Isolation
- Horizontal Scalability
- Enterprise API Gateway Pattern

5.2 System Architecture Diagram

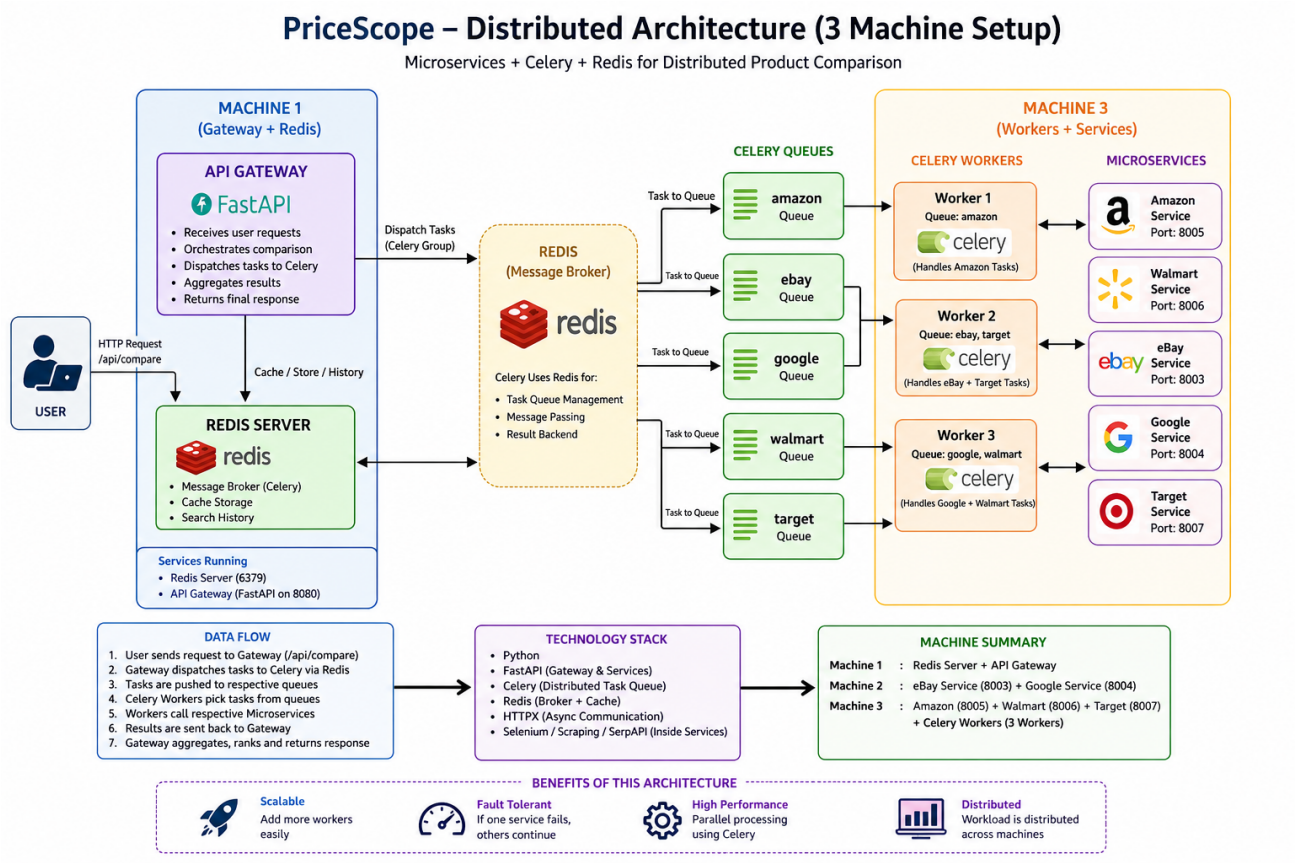

Figure 1: Enterprise Distributed Microservices Architecture of PriceScope showing API Gateway orchestration, distributed marketplace services, Redis-Celery distributed task execution, MCP intelligent orchestration, and asynchronous inter-service communication.

The architecture diagram illustrates the complete distributed workflow of the PriceScope platform. The frontend client sends requests to the centralized API Gateway, which asynchronously distributes workloads across multiple marketplace microservices including Amazon, eBay, Google Shopping, Walmart, and Target services. Redis and Celery manage distributed background execution and asynchronous task scheduling, while the MCP orchestration layer performs intelligent aggregation, semantic normalization, and ranking of retrieved product data.

6 Distributed Execution Workflow

1. User submits product query through frontend interface.
2. API Gateway initializes orchestration pipeline.
3. Gateway dispatches asynchronous requests concurrently to:
   - Amazon Service
   - eBay Service
   - Google Shopping Service
   - Walmart Service
   - Target Service
4. All marketplace services execute simultaneously using asynchronous distributed computation.
5. Each service independently performs:
   - Product extraction

- DOM parsing
- Price normalization
- Seller aggregation

6. MCP Server performs:
   - Semantic similarity matching
   - Product normalization
   - Intelligent ranking
   - Aggregation optimization
7. Redis coordinates distributed caching and messaging.
8. Celery workers execute distributed background tasks.
9. Final aggregated response is returned to frontend.

7 Distributed Computing Concepts Used

7.1 Microservices Architecture

Each marketplace service operates independently as an autonomous distributed component.

7.2 Asynchronous Processing

AsyncIO and HTTPX enable non-blocking asynchronous communication between services.

7.3 Parallel Execution

All marketplace services execute concurrently, significantly reducing latency.

7.4 Distributed Task Queue

Redis and Celery provide scalable distributed background task execution.

7.5 Fault Tolerance

Failure in one microservice does not terminate the complete system.

7.6 Horizontal Scalability

Each service can scale independently depending on computational demand.

7.7 Service Orchestration

API Gateway manages communication and aggregation between distributed services.

8 Technology Stack

Technology	Purpose
Python	Core backend programming language
FastAPI	High-performance asynchronous API framework
Redis	Distributed cache and message broker
Celery	Enterprise distributed task scheduling
HTTPX	Asynchronous HTTP communication
Selenium	Browser automation for dynamic scraping
BeautifulSoup	HTML parsing and extraction
MCP	Intelligent orchestration framework
AsyncIO	Non-blocking asynchronous execution
Uvicorn	ASGI server for FastAPI applications
HTML/CSS/JavaScript	Frontend visualization layer

Table 2: Technology Stack

9 Implementation Details

9.1 API Gateway

The API Gateway acts as the centralized orchestration layer responsible for:

- Request routing
- Parallel service dispatching
- Response aggregation
- Health monitoring
- Distributed communication management

9.2 Amazon Service

Amazon integration uses Selenium-powered browser automation for dynamic DOM extraction and JavaScript-rendered content retrieval.

9.3 Google Shopping Service

Google Shopping integration uses SerpAPI for scalable structured search retrieval.

9.4 eBay Service

eBay service performs asynchronous lightweight distributed product retrieval.

9.5 Walmart Service

Walmart integration combines scraping and distributed asynchronous retrieval mechanisms.

9.6 Target Service

Target integration performs scalable distributed marketplace aggregation and asynchronous product retrieval.

9.7 Redis and Celery

Redis acts as:

- Distributed cache
- Message broker
- Shared distributed queue

Celery enables:

- Distributed background workers
- Enterprise task scheduling
- Parallel workload distribution

10 Results and Performance Analysis

The system successfully demonstrates enterprise-grade distributed execution and asynchronous product aggregation.

10.1 Experimental Evaluation

Performance Parameter	Observed Status
Distributed Microservice Execution	Successful
Parallel Asynchronous Communication	Successful
Real-Time Product Aggregation	Operational
Independent Service Deployment	Stable
Fault Isolation Capability	Stable
Redis Distributed Messaging	Configurable
Celery Background Workers	Supported
Selenium Dynamic Scraping	Operational
MCP Intelligent Orchestration	Functional
Scalable API Gateway Routing	Successful
Concurrent Request Processing	Efficient
Low-Latency Distributed Execution	Achieved

Table 3: Distributed System Experimental Results

10.2 Observed Technical Outcomes

- Successful concurrent execution of all marketplace services.
- Real-time asynchronous product retrieval achieved.
- Distributed orchestration significantly reduced response latency.
- Service isolation improved system fault tolerance.
- Redis-ready distributed task infrastructure implemented.
- Celery-based enterprise task queue architecture integrated.
- MCP-based intelligent ranking and normalization operational.
- API Gateway successfully coordinated distributed communication.

11 Advantages

- Enterprise-grade distributed architecture
- Real-time parallel product retrieval
- Scalable microservices infrastructure
- Low-latency asynchronous communication
- Fault-tolerant system design
- Independent service deployment
- Efficient distributed orchestration
- Modular backend maintainability

12 Limitations

- Dynamic website structure changes
- External API rate limits
- Anti-bot restrictions
- Selenium browser memory overhead
- Dependency on third-party marketplaces

13 Future Enhancements

- Kubernetes-based orchestration
- Dockerized cloud-native deployment
- AI-powered recommendation engine
- Machine learning based price prediction
- Kafka-based distributed event streaming
- Real-time analytics dashboard
- GPU-accelerated ranking systems
- Load balancing and auto-scaling
- Multi-region distributed deployment
- Service mesh implementation using Istio

14 Conclusion

PriceScope successfully demonstrates the practical implementation of modern distributed computing principles using enterprise-grade technologies and scalable microservices architecture.

The platform integrates:

- Distributed asynchronous APIs
- Parallel service execution
- Redis-based distributed messaging
- Celery task scheduling
- Selenium automation
- MCP intelligent orchestration
- Real-time distributed aggregation

The architecture achieves scalable real-time product comparison while maintaining modularity, extensibility, scalability, and fault tolerance. The overall system closely resembles real-world enterprise cloud-native distributed platforms used in modern intelligent applications.

15 References

1. FastAPI Official Documentation
   https://fastapi.tiangolo.com/
2. Redis Documentation
   https://redis.io/documentation
3. Celery Distributed Task Queue
   https://docs.celeryq.dev/
4. Selenium Documentation
   https://www.selenium.dev/documentation/
5. HTTPX Async Client Documentation
   https://www.python-httpx.org/
6. MCP Documentation
   https://modelcontextprotocol.io/
7. Uvicorn ASGI Server
   https://www.uvicorn.org/
8. Designing Data-Intensive Applications
   Martin Kleppmann
9. Distributed Systems Principles and Paradigms
   Andrew S. Tanenbaum
10. Microservices Patterns
    Chris Richardson